MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2026/2027

LAB REPORT 2

MCTA 3203

SECTION 1

GROUP 8

NO.	MATRIC NO.	NAME
1	2318483	SYED MOHAMAD SYAFIEE BIN SYED TAJUDIN
2	2413869	SHARIF NUR AZWAR BIN SHARIF ABU BAKAR
3	2413733	ZAHIN ZAYDAN BIN ZAIRY

SUBMISSION DATE : 19 OCTOBER 2026

Abstract

This week's experiments focused on establishing serial communication between an Arduino UNO and Python using PyCharm to understand the interaction between hardware and software in real-time applications. The experiments also aimed to explore how a potentiometer can be used to control electronic components by adjusting the analog-to-digital converter (ADC) input values. Two main tasks were carried out during the experiment. In Task 1, the analog readings from the potentiometer were collected by the Arduino UNO and transmitted to Python through serial communication. The received values were then used to control the LED based on the input readings. This task demonstrated how data could be transferred between hardware and software to perform a simple control function. In Task 2, the potentiometer was used to adjust the angular position of a servo motor through serial communication between the Arduino UNO and Python. The servo motor responded to changes in the potentiometer input, while Python displayed the motor's angular position in real time through a plot. Overall, these experiments provided practical experience in programming, serial communication, and hardware integration. They also enhanced the understanding of how sensors, actuators, and software interfaces work together in a mechatronic system, particularly in real-time monitoring and control applications.

Table of Content

Abstract

1. Introduction

2. Material and Equipment

3. Experiment Setup

4. Methodology

5. Data Collection

6. Data Analysis

7. Result

8. Conclusion

9. Recommendations

10. References

11. Appendices

12. Acknowledgement

13. Student Declaration

WEEK 3: SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

1. Introduction

This experiment investigates the integration of hardware and software by establishing serial communication between an Arduino UNO and Python. Serial communication enables data exchange between a microcontroller and a computer, allowing electronic components to be controlled and monitored in real time. The main hardware components used in this experiment are the Arduino UNO, potentiometer, LED, and servo motor. Meanwhile, the Arduino IDE is used to develop and upload code to the microcontroller, while PyCharm is used to execute Python scripts.

In Task 1, the Arduino UNO reads the analog voltage values generated by the potentiometer and transmits the data to Python through serial communication. Based on the received input values, the LED is controlled according to the specified conditions. This task demonstrates how input signals can be processed and used to control an output device through communication between hardware and software.

The potentiometer configuration used in this experiment is illustrated in **Figure 1.0** below:

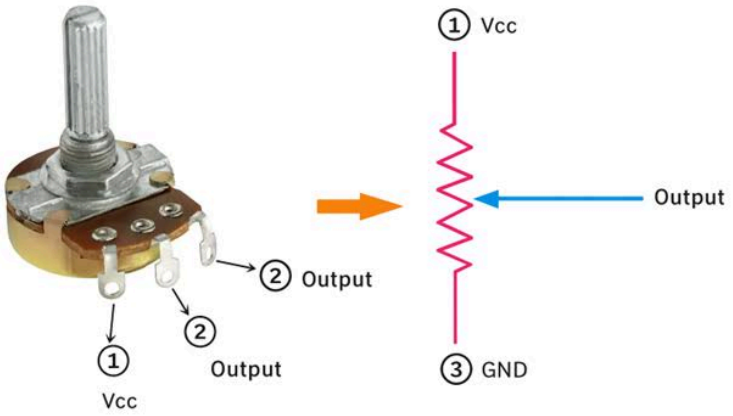


Figure 1.0 Shows the potentiometer pinout

In Task 2, the potentiometer is used as an input device to control the angular position of a

servo motor. When the potentiometer is rotated, the analog input value changes and is processed by the Arduino before being transmitted to Python through serial communication. Python then uses the received data to monitor and display the servo motor's angular position in real time.

The main objectives of this experiment were:

- To control an LED by adjusting the potentiometer and transmitting the input values from the Arduino to Python through USB serial communication.
- To control the angular position of a servo motor in real time using input values from the potentiometer.
- To implement a function that allows the program to be terminated using a designated keyboard key.
- To use the Matplotlib library in Python to visualize and plot potentiometer readings or servo motor position data in real time.

2. Materials and Equipments

- Cytron CT-UNO 1
- Potentiometer 1
- Light Emitting Diode, LED 1
- Servo Motor 1
- 220 Ω Resistor 1
- Breadboard 1
- Jumper wires

3. Experimental Setup

3.1 Task 1

The potentiometer was connected to the Arduino UNO using a breadboard. One of its outer terminals was connected to the common 5V rail, while the other was connected to the ground (GND) rail. The middle terminal was connected to the analog input pin A0. For the LED circuit, the anode was connected to digital pin 7 through a 220 Ω resistor, while the cathode was connected to the common ground rail.The Arduino transmitted the analog readings from the potentiometer to Python in PyCharm through serial communication. Based on the received values, the LED was switched ON or OFF according to the potentiometer position and the conditions defined in the program. The circuit configuration is illustrated below.

Circuit Connection:

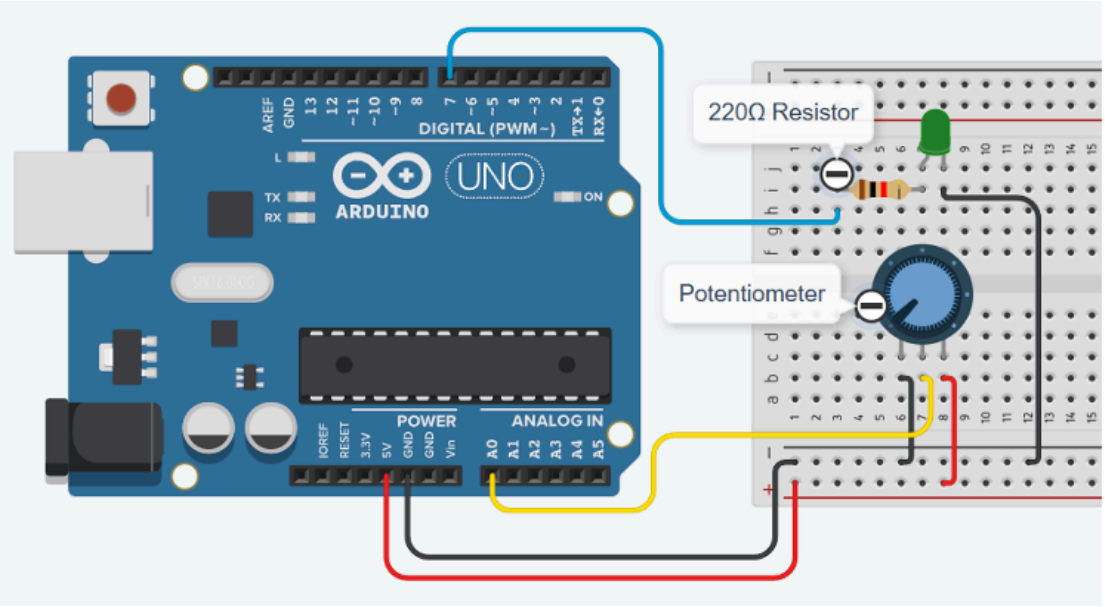


Figure 2 Shows the schematic diagram for Task 1 using Tinkercad

When the potentiometer is rotated and its reading exceeds half of the maximum value, the LED will switch ON.

3.2 Task 2

The servo motor's signal pin was connected to digital pin 9 of the Arduino UNO, while its power and ground wires were connected to the 5V and GND pins, respectively. The Arduino received angle values from Python through serial communication and adjusted the servo motor's position accordingly. A potentiometer and an LED were then incorporated into the circuit, allowing the servo motor's angular position to be adjusted in real time by rotating the potentiometer.

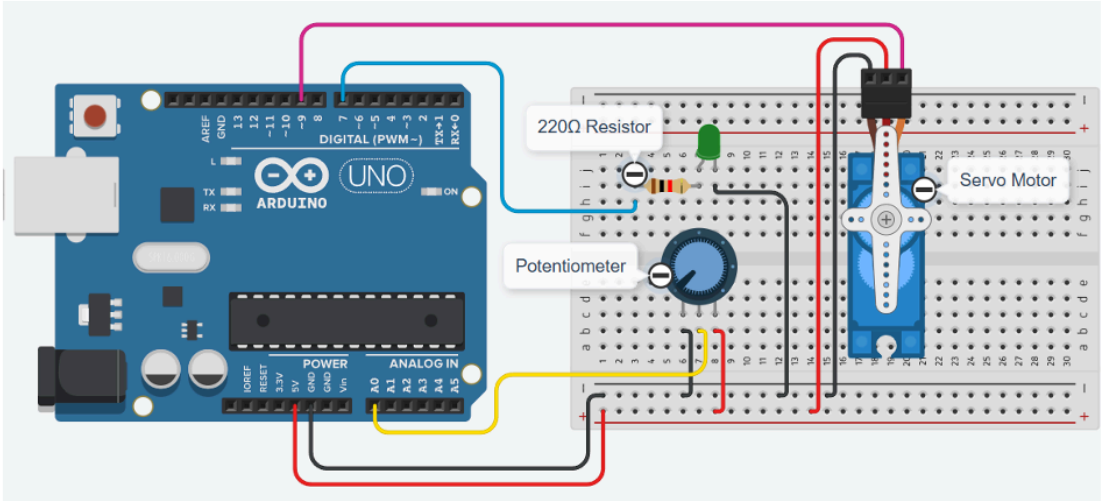


Figure 3 Shows the schematic diagram for Task 2 using Tinkercad

- The Arduino reads the analog values from the potentiometer and adjusts the servo motor's angular position accordingly.
- The servo motor's angle data is transmitted from the Arduino to Python through serial communication.
- An LED is incorporated into the circuit to provide visual feedback by turning on or adjusting its brightness according to the potentiometer reading or servo position.
- A keyboard control function is implemented to allow the program to be terminated safely during execution.

4. Methodology

4.1 Task 1

4.1.1 Circuit Assembly

All components were connected using a breadboard according to the circuit configuration shown in Figure 2. The potentiometer was connected to the Arduino's analog input pin A0, while the LED was connected to digital output pin 7 through a 220 Ω resistor to limit the current. The Arduino was powered through a USB connection to the computer, supplying the required 5 V operating voltage to the circuit. Jumper wires were used to establish the electrical connections between the components.

4.1.2 Serial Communication Setup

The Arduino IDE was used to write and upload the program to the Arduino UNO. The program read analog input values from the potentiometer through pin A0 and compared the readings with a predefined threshold of 500. If the potentiometer reading exceeded 500, the LED connected to digital pin 7 was switched ON; otherwise, the LED remained OFF. The Arduino continuously transmitted the potentiometer readings to Python through serial communication at a baud rate of 9600 using the COM3 port. Python could then receive and display the transmitted data for monitoring purposes. A delay of 100 milliseconds was included between readings to ensure stable operation. This setup demonstrated how potentiometer input values could be used to control an LED through Arduino programming and serial communication.

4.1.2.1 Code Upload to Arduino IDE

```
const int potPin = A0; // Analog input pin connected to the potentiometer

const int ledPin = 8; // Digital output pin connected to the LED with a 220 ohm resistor



void setup() {

Serial.begin(9600); // Initialize serial communication at 9600 baud rate

pinMode(ledPin, OUTPUT); // Set the LED pin as an output

}
```

```
void loop() {

// Read the potentiometer value (range: 0 to 1023) and send it over the serial port

int potValue = analogRead(potPin);

Serial.println(potValue);



// Check if there is an incoming command from the Python script

if (Serial.available() > 0) {

char command = Serial.read();

if (command == '1') {

digitalWrite(ledPin, HIGH); // Turn LED ON

} else if (command == '0') {

digitalWrite(ledPin, LOW); // Turn LED OFF

}

}



delay(200); // Add a short delay to avoid sending data too fast

}
```

4.1.3 Serial Plotter

The Arduino Serial Plotter was used to visualize the potentiometer readings in real time. First, the correct COM port (COM3) was selected in the Arduino IDE, and the baud rate was set to 9600 to match the value specified in the Arduino program. As the potentiometer knob was rotated, the Serial Plotter displayed the changing analog readings as a graph. The graph settings, such as the display range, labels, and colors, could be adjusted when necessary.

Meanwhile, the LED switched ON or OFF automatically depending on whether the potentiometer reading exceeded the predefined threshold of 500, providing immediate visual feedback. The Serial Plotter can be accessed by selecting Tools → Serial Plotter in the Arduino IDE. However, it was closed whenever Python needed to access the serial port to prevent communication conflicts between the two applications.

4.1.4. Python Script

```
import serial

import time

# Establish serial communication on port COM5 at 9600 baud rate

ser = serial.Serial('COM5', 9600, timeout=1)

time.sleep(2)  # Wait for the serial connection to initialize


try:

    while True:

        # Read data from Arduino and decode it

        raw_data = ser.readline().decode('utf-8', errors='ignore').strip()


        if raw_data.isdigit():

            pot_value = int(raw_data)


            # Turn LED ON if potentiometer value exceeds half of its maximum range (512)

            if pot_value > 512:

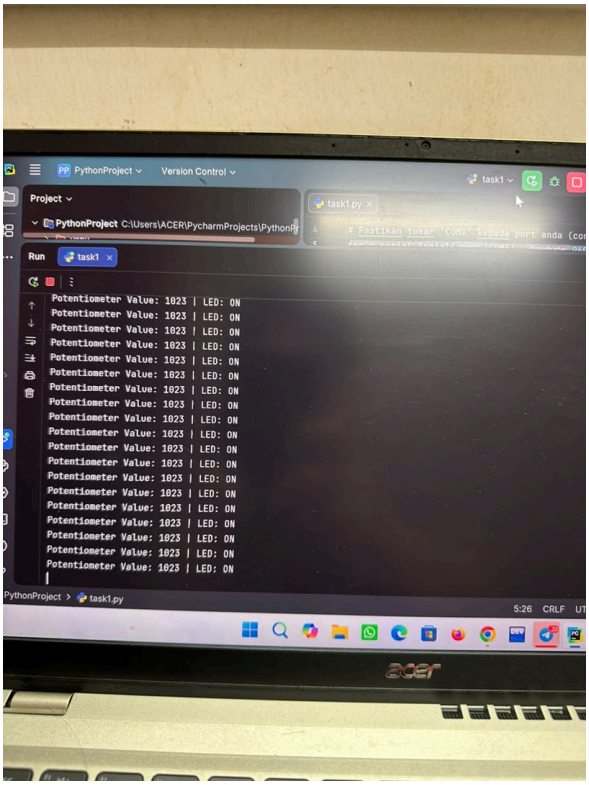
                ser.write(b'1')  # Send command to turn LED ON
```

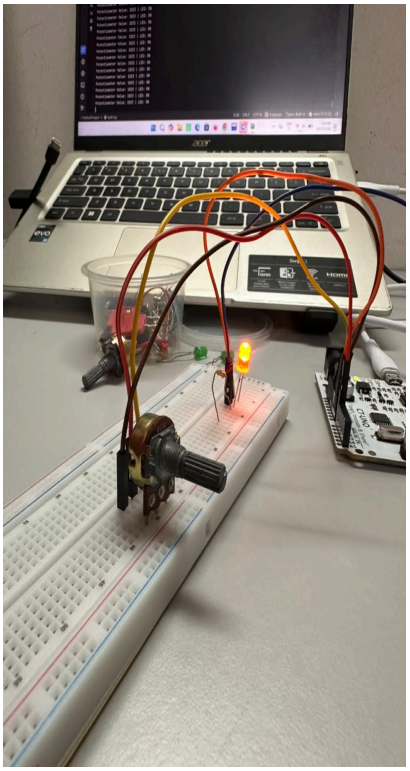
```
            led_status = "ON"
        else:
            ser.write(b'0')  # Send command to turn LED OFF
            led_status = "OFF"

        print(f"Potentiometer Value: {pot_value} | LED Status: {led_status}")

except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
```

4.1.4.1 Code Upload to PyCharm





4.2 Task 2

4.2.1 Circuit Assembly

All components were connected on a breadboard according to Figure 3. The potentiometer was also used to control the servo motor. The servo's signal pin was connected to Arduino digital pin 9, while its VCC and GND pins were connected to 5 V and GND, respectively. The LED from Task 1 was reused to provide visual feedback corresponding to the potentiometer input or servo position.

4.2.2 Code Upload to Arduino IDE

The Arduino IDE was used to develop and upload the enhanced code to the Arduino. The potentiometer input was mapped to a servo angle ranging from 0° to 180°, and the angle was continuously transmitted to Python. The LED was also updated to provide visual feedback based on the potentiometer input or servo position. Additionally, a keyboard control feature was implemented to safely stop Arduino execution when prompted:

```
#include <Servo.h>

Servo myservo;

const int potPin = A0;   // Potentiometer pin

const int ledPin = 7;    // LED pin

const int servoPin = 9;  // Servo pin

bool isRunning = true;

void setup() {

  Serial.begin(9600);

  myservo.attach(servoPin);

  pinMode(ledPin, OUTPUT);

}

void loop() {

  // Stop if 'q' command is received from the computer
```

```
 if (Serial.available() > 0) {
   char key = Serial.read();
   if (key == 'q' || key == 'Q') {
     isRunning = false;
     myservo.write(90);          // Stop 360 servo
     digitalWrite(ledPin, LOW);  // Turn off LED
   }
 }
 if (isRunning) {
   int potValue = analogRead(potPin);             // Read potentiometer (0 - 1023)
   int servoPos = map(potValue, 0, 1023, 0, 180); // Convert to 0 - 180 scale
   myservo.write(servoPos);                       // Move servo
   // Turn on LED if potentiometer is greater than half (> 512)
   if (potValue > 512) {
     digitalWrite(ledPin, HIGH);
   } else {
     digitalWrite(ledPin, LOW);
   }
   Serial.println(potValue); // Send value to Python
   delay(100);
 }
```

4.2.3. Python Script for Servo and Visualization at PyCharm

Python received the servo angle data through the serial connection and displayed it in real time using Matplotlib. Interactive plotting enabled the servo's movement to be monitored while the potentiometer was adjusted. Once the experiment was completed, the serial connection was safely closed:

```
import serial
import matplotlib.pyplot as plt
import time

ser = serial.Serial('COM5', 9600, timeout=1)
time.sleep(2)

plt.ion()
fig, ax = plt.subplots()
x_vals, y_vals = [], []

try:
    while True:
        pot_value = ser.readline().decode('utf-8', errors='ignore').strip()
        if pot_value.isdigit():
            val = int(pot_value)
            print("Potentiometer Value:", val)

            x_vals.append(len(x_vals))
            y_vals.append(val)

            ax.clear()
            ax.plot(x_vals, y_vals)
            plt.pause(0.1)

except KeyboardInterrupt:
    pass
finally:
    ser.write(b'q')
    ser.close()
    plt.ioff()
    plt.show()
    print("Serial connection closed.")
```

5. DATA COLLECTION

Task 1 and Task 2

The result was made clearly by observation through our own eyes

1. The LED lights up when the value on the serial monitor is higher than 512.
2. The LED turns off when the value is lower than 512.

6. DATA ANALYSIS

Task 1 and Task 2

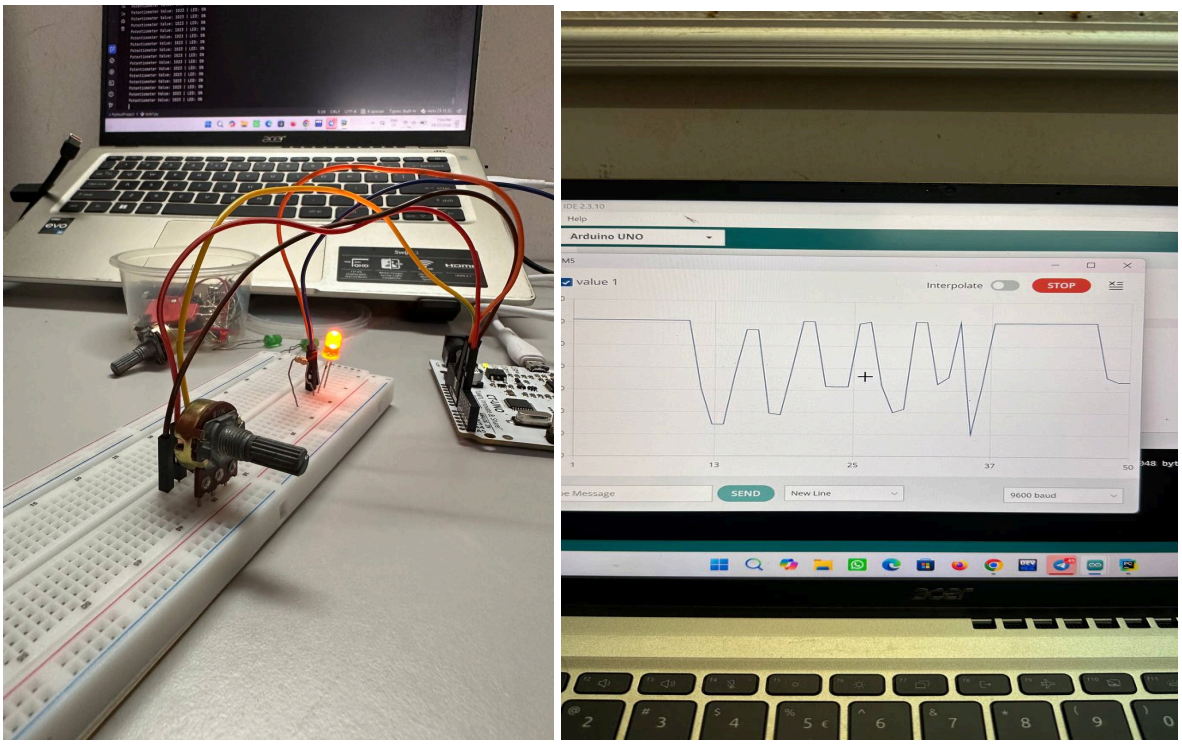
The data confirmed that:

1. The potentiometer and the LED are working properly.
2. The python logic controls the LED condition instead of the Arduino IDE.

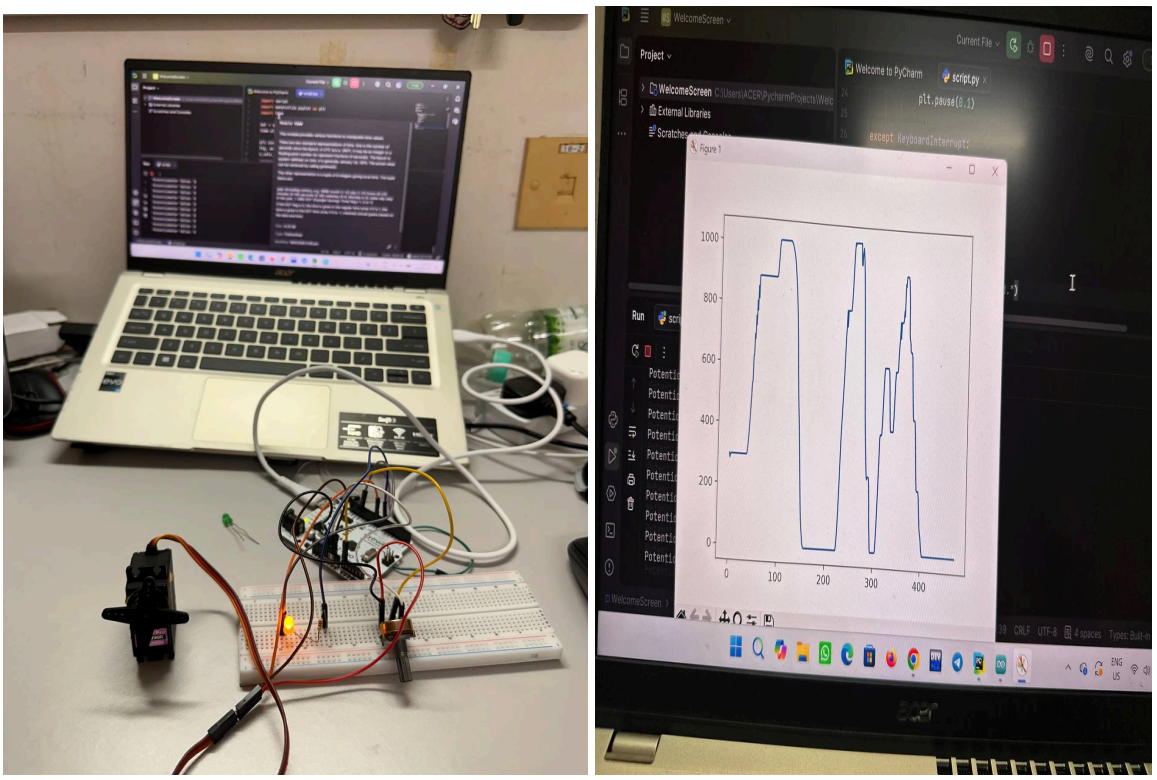
7. RESULT AND ENGINEERING DISCUSSION

Based on our hardware photos and screenshots from both the Arduino Serial Plotter and PyCharm, we successfully established real-time serial communication between the Cytron CT-UNO and Python at a 9600 baud rate. The physical hardware behaved just as expected from our initial hypotheses: turning the potentiometer gave smooth ADC readings from 0 to 1023 on the Serial Plotter and Matplotlib graph , the LED turned on right after passing the midpoint value of 512, and the servo motor rotated smoothly between 0° and 180° before resetting to 90° whenever we pressed to stop the program. However, we did run into a few common issues like COM port conflicts, slight signal noise, and plotting lag, which we fixed through simple adjustments in our setup and code. For instance, we made sure to close the Arduino Serial Plotter before running the PyCharm script so they wouldn't fight over the same port, and we added time.sleep(2) along with ser.close() in a finally block to handle the connection cleanly. To prevent the script from crashing due to random serial noise or incomplete data, we filtered the incoming strings using errors='ignore' and .isdigit(), while matching the Arduino's delay(100) with Python's plt.pause(0.1) kept the live graph responsive without any noticeable drift.

Throughout the experiment, we encountered a few technical errors and challenges, which were successfully resolved using systemic workarounds. One major issue was the COM port detection conflict caused by the Arduino Serial Plotter attempting to access the same serial port simultaneously with the Python script, which we solved by ensuring the Serial Plotter was completely closed before running PyCharm, alongside adding time.sleep(2) and a finally block (ser.close()) to handle the serial connection cleanly. Additionally, potential data corruption and crashes caused by physical electrical signal noise or incomplete serial bytes were mitigated by filtering the incoming data strings using errors='ignore' and .isdigit(). Lastly, plotting lag or timing drift between the physical potentiometer adjustments and the graph rendering was eliminated by matching the Arduino's transmission delay (delay(100)) with Python's Matplotlib pause (plt.pause(0.1)), ensuring smooth synchronization between the hardware action and software visualization.



Photographic evidence for task 1



Photographic evidence for task 2

7. Conclusion

Task 1 (Potentiometer & Python-Controlled LED)

In Task 1, analog voltage signals from a potentiometer were read across the full ADC range of 0 to 1023 on pin A0 and transmitted to Python. Instead of relying on onboard Arduino logic, decision-making was handled entirely on the host computer: Python evaluated the incoming values against a midpoint threshold of 512 and sent serial commands ('1' or '0') back to switch the LED ON or OFF in real time.

Task 2 (Servo Motor Control, Feedback & Real-Time Visualization)

In Task 2, potentiometer inputs were mapped from the 0–1023 raw scale to a 0°–180° angular scale to control a servo motor on digital pin 9. Telemetry was streamed simultaneously to Python and plotted dynamically using Matplotlib (plt.ion()), while secondary LED visual feedback was maintained when readings exceeded 512. Finally, the system incorporated a clean shutdown mechanism where sending a 'q' command via Python reset the servo to 90°, extinguished the LED, and safely closed the serial port connection.

8. Recommendations

Ensure any monitoring tools such as Arduino Serial Monitor or Serial Plotter are completely closed before executing the python script to prevent port-access collisions. Also, using same servo motor would help avoiding unorthodox outcomes due to coding

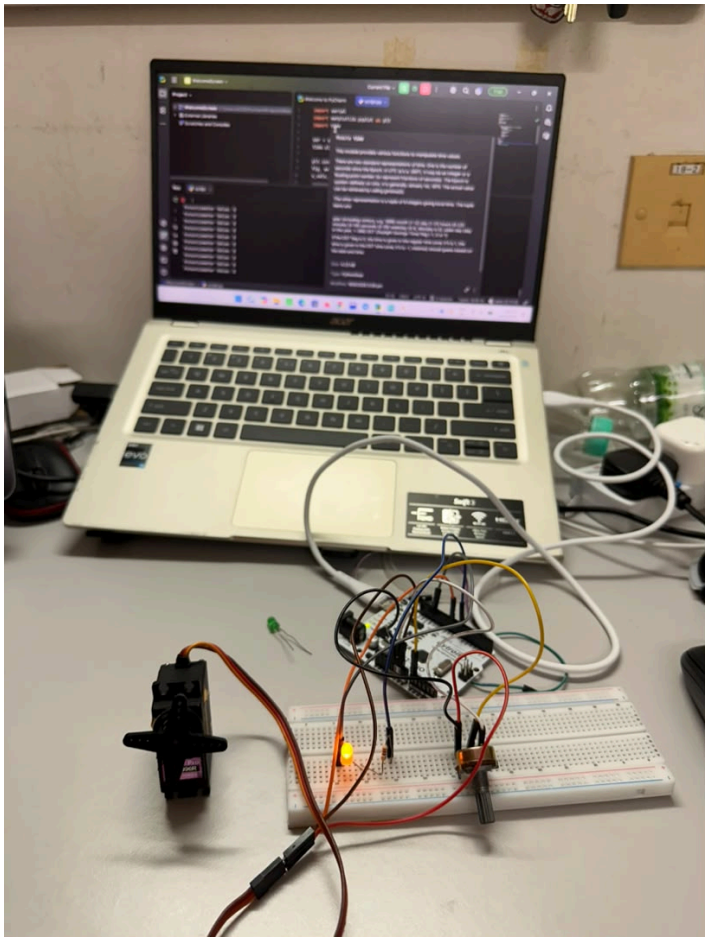
9. Appendices

10. References

7 Types of servo motor: An overview of different varieties & their applications. (2022, May 18). https://assunmotor.com/blog/types-of-servo-motors/

11. VIDEO SIMULATION

https://drive.google.com/drive/folders/10gBdMW5wtZZ-xRBlBmaCXuWPWTVMy2ZP?usp=drive_link



ACKNOWLEDGEMENTS

Alhamdulillah, all praise and gratitude be to Allah SWT for His blessings, guidance, and mercy that enabled us to successfully complete this experiment.Throughout the development process, we faced several challenges, but with His will and guidance, we managed to overcome them and complete the project successfully. We would also like to express our heartfelt appreciation to our lecturer for their continuous guidance, support, and valuable feedback throughout this project. Their advice has greatly helped us in understanding both the theoretical and practical of digital logic systems and practical hardware interfacing using a Cytron CT-UNO microcontroller, Potentiometer and Servo Motor

Student's Declaration

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.We therefore, agreed unanimously that this report shall be submitted for marking and this final The printed report has been verified by us.

Signature: *sharif* Read [/]

Name: Sharif Nur Azwar Bin Sharif Abu Bakar Understand [/]

Matric Number: 2413869 Agree [/]

Signature: *syed*

Name: Syed Mohammad Syafiee Bin Syed Tajudin

Matric Number:2318483

Read [/]

Understand [/]

Agree [/]

Signature: *zahin*

Name: Zahin Zaydan bin Zairy

Matric Number:2412733

Read [/]

Understand [/]

Agree [/]